

Módulo conversor entre archivos y cadenas de bytes

Documentación por Thiago Grillo

Este documento pretende explicar y servir como instructivo, justificación, y documentación sobre el módulo conversor que se utiliza para la demostración del uso del Proyecto Col-Fi, integrado y desarrollado por Ciro Lopez, Yago Sayas, Thiago Grillo, Lucas Marin y Thiago Puga. Este programa fue desarrollado por Thiago Grillo, y aquí documentado para su comprensión y correcta utilización .

Índice

- 1. Propósito y Argumentación.**
- 2. Código fuente y aclaraciones.**
- 3. Explicación y Técnicas.**
- 4. Interfaz Gráfica del Usuario.**
- 5. Actualización Global.**
- 6. Conclusión y Cierre.**

1. Propósito y Argumentación

A la hora de realizar el proyecto principal Col-Fi, necesitamos una forma ágil, práctica y sencilla de transformar los archivos a **cadenas de bytes**. Para fines prácticos y demostrativos, este proyecto solo transforma archivos de tipo PDF a cadenas de bytes (8 bits). Es fructífero resaltar que esto es una herramienta y extensión del proyecto principal “Col-Fi”.

Este proyecto está hecho para transmitir información tal como lo hacemos día a día en internet, donde descargamos y subimos archivos a la web, por lo tanto no queremos solo limitarnos a hacer “un traductor de caracteres a binario”, sino demostrar que es una forma válida y funcional de transmitir datos de tal tamaño semejantes a un archivo PDF, formato popular hoy en día en materia de envío de información, presentaciones, textos explicativos, entre otros.

Habiendo planteado el problema y respondido a las necesidades del proyecto, en las siguientes secciones, explicaremos cómo funciona este módulo.

2. Código fuente y aclaraciones

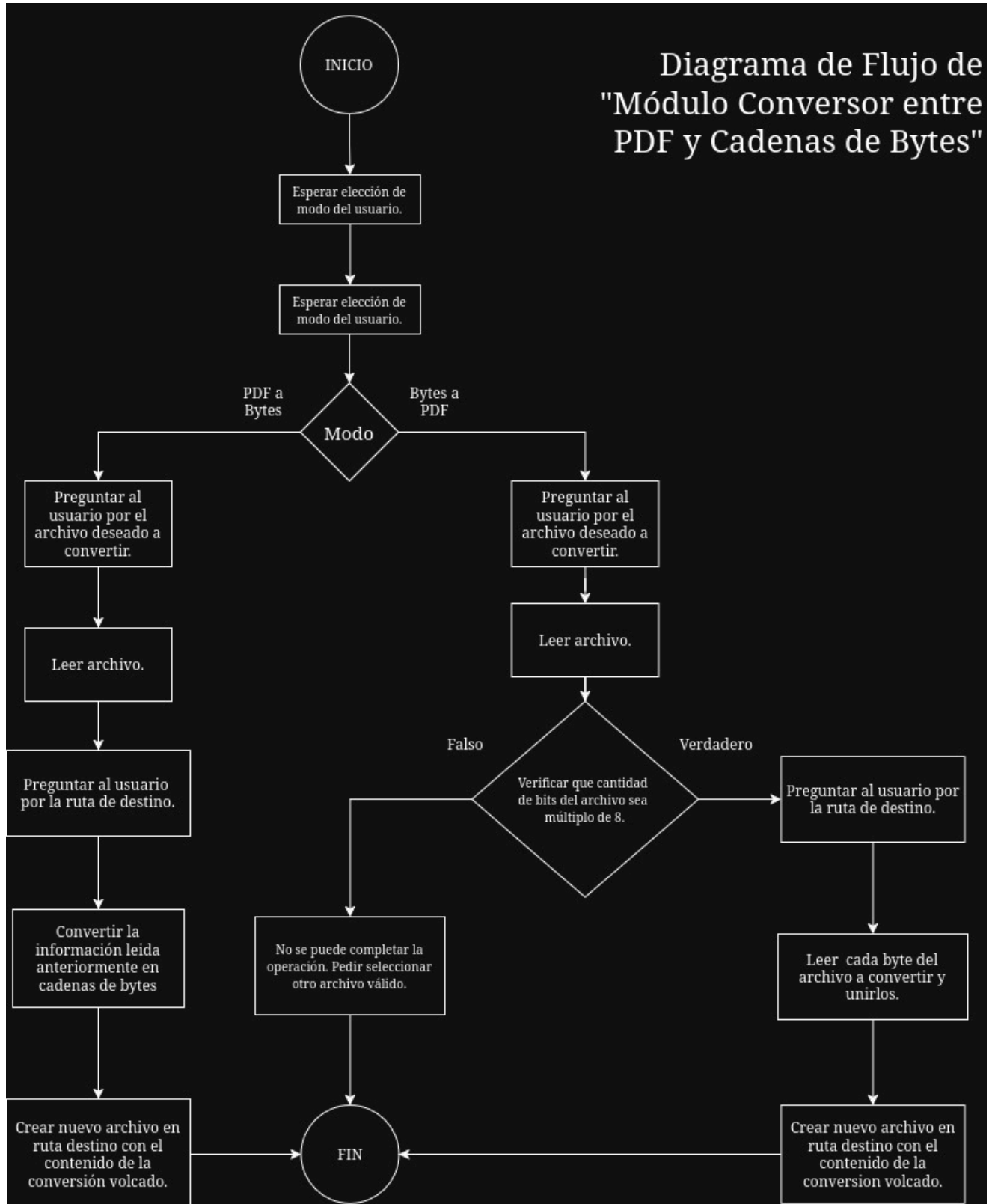
El código de este programa fue desarrollado enteramente en Python, utilizando las librerías OS, time y Tkinter (apartado gráfico incluyendo módulos extra: messagebox y filedialog).

Se divide en funciones y contiene un selector de modo para elegir qué operación realizar.

En las siguientes páginas, se incluye el diagrama de flujo explicativo sobre el proceso principal del programa y cómo funciona lógicamente y el código fuente.

DIAGRAMA DE FLUJO:

Diagrama de Flujo de "Módulo Conversor entre PDF y Cadenas de Bytes"



CÓDIGO FUENTE:

```

from tkinter import filedialog
from tkinter import messagebox
import os
import tkinter as tk
import webbrowser
import time

root = tk.Tk()
root.geometry("500x300")
root.resizable(False, False)
root.title("colFi. - Conversor de PDF a bytes")
#root.attributes("-toolwindow", True)

root.config(bg="#111212")
modo = tk.IntVar(value=0)

def pdf_to_bits():
    """
    Convierte un PDF a una secuencia de bits (8 bits por byte)
    """
    try:
        pdf = filedialog.askopenfilename(title="Selecciona un PDF",
filetypes=(("Archivo PDF", "*.pdf"),))
        log(f"Archivo seleccionado: {pdf}")
        with open(pdf, "rb") as f:
            data = f.read()
            log(f'Leyendo archivo...')

            filename = os.path.splitext(os.path.basename(pdf))[0]
            log(f"Iniciando conversión de {filename}...")
    except Exception as e:
        log("-----")
        log(f"ERROR: Hubo un error o la operación fue cancelada
({e}).")
        messagebox.showerror("Error", "ERROR: Hubo un error o la
operacion fue cancelada ({e})")
        return

    log("Esperando por la ruta de destino...")
    try:

```

```

        path = filedialog.askdirectory(title="Selecciona la ruta de
destino")
        bits = ''.join(f'{byte:08b}' for byte in data)
        log(f"Se generaron {len(bits)} bits")
        log("Creando archivo de cadena de bytes...")
        output_path = os.path.join(path, f"{filename}.txt")
        with open(output_path, "w") as f:
            f.write(bits)
    except Exception as e:
        log("-----")
        log(f"ERROR: Hubo un error o la operacion fue cancelada
({e})")
        messagebox.showerror("Error", f"ERROR: Hubo un error o la
operacion fue cancelada ({e})")
        return

    log("La operación finalizó con éxito.")
    log("-----")
    messagebox.showinfo("Operacion exitosa", f"Se convirtió el
archivo con éxito. Guardado en {path}")

def bits_to_pdf():
    """
    Reconstruye un PDF desde una secuencia de bits
    """
    try:
        txt_path = filedialog.askopenfilename(title="Selecciona un
texto", filetypes=(("Archivo texto", "*.txt"),))
        log(f"Archivo seleccionado: {txt_path}")
        with open(txt_path, "r") as f:
            bits = f.read().strip()
            log(f'Leyendo archivo...')

            filename = os.path.splitext(os.path.basename(txt_path))[0]
    except Exception as e:
        log("-----")
        log(f"ERROR: Hubo un error o la operacion fue cancelada
({e})")
        messagebox.showerror("Error", f"ERROR: Hubo un error o la
operacion fue cancelada ({e})")
        return

```

```

# Verificación
log(f"Verificando {filename}...")
try:
    if len(bits) % 8 != 0:
        raise ValueError("La cantidad de bits no es múltiplo de
8.")
except ValueError as e:
    log("-----")
    log(f"ERROR: {e}")
    messagebox.showerror("Error", f"ERROR: Hubo un error o la
operacion fue cancelada ({e})")
    return

log("Esperando por la ruta de destino...")
try:
    path = filedialog.askdirectory(title="Selecciona la ruta de
destino")
    log(f"Iniciando conversión de {filename}...")
    bytes_recuperados = bytearray()

    for i in range(0, len(bits), 8):
        byte = bits[i:i+8]
        bytes_recuperados.append(int(byte, 2))

    log(f"Se recuperaron {len(bytes_recuperados)} bytes")
    log("Creando archivo...")
    output_path = os.path.join(path, f"{filename}.pdf")
    with open(output_path, "wb") as f:
        f.write(bytes_recuperados)
except Exception as e:
    log("-----")
    log(f"ERROR: Hubo un error o la operacion fue cancelada
({e})")
    messagebox.showerror("Error", f"ERROR: Hubo un error o la
operacion fue cancelada ({e})")

log("La operación finalizó con éxito.")
log("-----")
messagebox.showinfo("Operacion exitosa", f"Se convirtió el
archivo con éxito. Guardado en {path}")

```

```

consola = tk.Text(
    root,
    height=8.5,
    width=67,
    bg="#0d0d0d",
    fg="#00ff88",
    state='disabled',
    wrap="word",
    insertbackground="white",  # color del cursor
    borderwidth=1,
    font=("Consolas", 9)
)
consola.place(x=12, y=164)

scroll = tk.Scrollbar(root)
scroll.place(x=480, y=164, height=130)
scroll.config(command=consola.yview)

def log(text):
    global consola
    tm = time.strftime("[%H:%M:%S]")
    consola.config(state="normal")
    consola.insert(tk.END, f'{tm} {text}\n')
    consola.see(tk.END)
    consola.config(state="disabled")

def start_conversion():
    global modo, consola
    consola.config(state="normal")
    consola.insert(tk.END, " \n")
    consola.config(state="disabled")
    if (modo.get() == 0):
        log("--- PDF a Bytes ---")
        log("Esperando por un archivo PDF...")
        pdf_to_bits()
    elif (modo.get() == 1):
        log("--- Bytes a PDF ---")
        log("Esperando por un archivo de cadena de bytes...")
        bits_to_pdf()
    else:

```



```
log("ERROR!")
```

```
logo_text = tk.Label(  
    root,  
    text="colFi. -----",  
    foreground="#fff",  
    background="#111212",  
    cursor="hand2",  
    font=('Arial', 28, 'italic bold')  
)  
logo_text.place(x=9, y=5)  
def open_page(event):  
    webbrowser.open("https://col-fi.netlify.app/")  
logo_text.bind("<Button-1>", open_page)
```

```
select_text = tk.Label(  
    root,  
    text="Seleccione su operacion",  
    foreground="#fff",  
    background="#111212",  
    font=('Arial', 9)  
)  
select_text.place(x= 185, y=55)
```

```
tk.Radiobutton(  
    root,  
    text="PDF a Bytes",  
    variable=modo,  
    value=0,  
    bg="#111212",  
    fg="white",  
    selectcolor="#2b2b2b", # color del círculo seleccionado  
    activebackground="#111212",  
    activeforeground="white",  
    highlightthickness=0  
) .place(x=155, y=75)  
tk.Radiobutton(  
    root,  
    text="Bytes a PDF",  
    variable=modo,  
    value=1,
```

```

        bg="#111212",
        fg="white",
        selectcolor="#2b2b2b",
        activebackground="#111212",
        activeforeground="white",
        highlightthickness=0
    ).place(x=255, y=75)

# CONVERT BUTTON
boton_convertir = tk.Button(
    root,
    text="Convertir",
    bg="#2b2b2b",
    fg="white",
    activebackground="#3a3a3a",
    activeforeground="white",
    borderwidth=0,
    padx=20,
    pady=10,
    cursor="hand2",
    command=start_conversion
)
boton_convertir.place(x=205, y= 110)

def open_gh(event):
    webbrowser.open("https://github.com/gr1ll0o/proyect-colfi.git")

github = tk.Label(
    root,
    text="GitHub",
    fg="blue",
    bg="#111212",
    cursor="hand2",
    font=("Arial", 12, "underline")
)

github.place(x=425, y=4)
github.bind("<Button-1>", open_gh)

log('¡Pulsa el logo para ver la página del proyecto!')
log('Seleccione el modo deseado y presione "Convertir".')

```

```
if __name__ == "__main__":  
    root.mainloop()
```

(Código disponible en el [repositorio del proyecto principal Col-Fi](#))

3. Explicación y técnicas.

El código se divide en dos funciones principales las cuales hacen la conversión: **pdf_to_bits()** y **bits_to_pdf()**

La primera función lee el contenido del archivo seleccionado (el cual el usuario selecciona a través de **filedialog.askopenfilename**).

Acto seguido convierte todo ese contenido en secuencias de 8 dígitos, o sea cadenas de bytes. Guarda ese contenido transformado en un nuevo archivo como output. Este es el producto deseado, y propósito de este módulo.

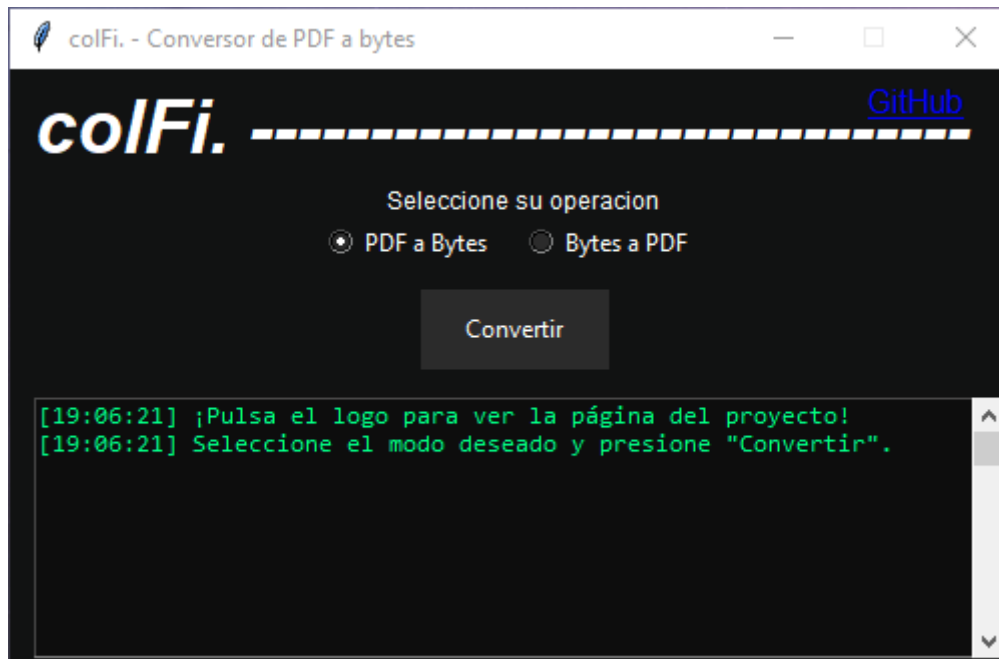
La segunda función hace lo contrario: se encarga de reconstruir un PDF a través de un archivo de cadenas de bytes (preferentemente generado por este programa claro). Esta es la función predilecta del receptor en el proceso de recibir la información en bits de un emisor. Gracias a esta función convertiremos esos datos crudos y binarios a información valiosa y útil, de vuelta en un formato legible como PDF.

Esta función cuenta con una comprobación extra e importante, la cual es comprobar que el número de bits en el archivo importado sea múltiplo de 8, de esta forma garantizamos que la transformación respete la integridad de una cadena válida de bits, dicho de otra manera, con esto verificamos que toda la información esté completa, válida y apta para reconstruirse en un archivo PDF. El código reconstruido se guarda en un nuevo archivo PDF si es que el proceso se completó satisfactoriamente.

4. Interfaz Gráfica del Usuario

1. Aspecto General.

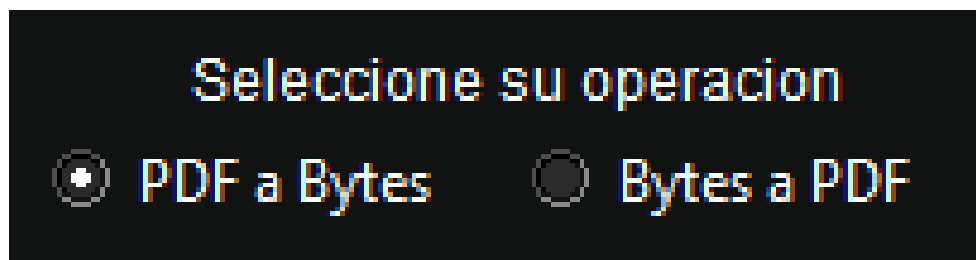
Como se explicó en el capítulo anterior, para formar la GUI del algoritmo se utilizó la librería nativa de Python, Tkinter.



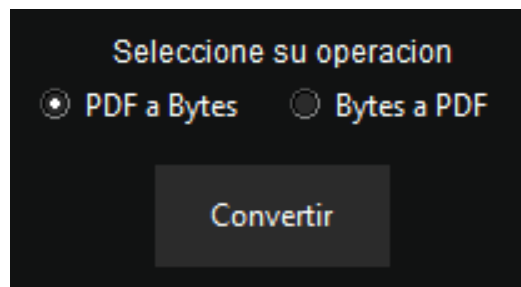
2. Selección de Modo.

La interfaz es sencilla, estética e intuitiva. El usuario selecciona el modo deseado a partir de 2 radio buttons, solo uno puede estar activo a la vez:

- **MODO 1:** PDF a Bytes
- **MODO 2:** Bytes a PDF



3. Botón “Convertir”.



Una vez seleccionado el modo deseado, el programa aguarda a que se pulse el botón “Convertir”.

Dependiendo del modo seleccionado por los radio button, el comportamiento del programa cambia. De forma general, se le solicita al usuario seleccionar el archivo a convertir.

Si el **MODO 1** está seleccionado, el programa solo admitirá archivos con extensión ‘.pdf’.

En caso contrario, si el **MODO 2** está seleccionado, el programa solo admitirá archivos con extensión ‘.txt’. Claramente estos archivos de texto deben cumplir con el formato esperado por el programa: cadenas de bytes exactas, sin déficit de caracteres.

4. Consola Informativa

Como última e interesante función, el software incluye una pequeña consola de control que permite ver en tiempo real qué está sucediendo, el estado del programa, y en caso de errores la posibilidad de ver de forma clara qué falló en el proceso de conversión *—filosofía importante a lo largo de todo el proyecto Col-Fi, la capacidad de analizar y corregir errores—*.

```
[19:22:32] Esperando por un archivo PDF...
[19:22:34] Archivo seleccionado: C:/Users/GRIILL0/Desktop/Ficha de
salud 2026.pdf
[19:22:34] Leyendo archivo...
[19:22:34] Iniciando conversión de Ficha de salud 2026...
[19:22:34] Esperando por la ruta de destino...
[19:22:37] Se generaron 1583584 bits
[19:22:37] Creando archivo de cadena de bytes...
[19:22:37] La operación finalizó con éxito.
```

5. Actualización Global.

Recientemente, específicamente el día 11 de junio de 2026, el software recibió una importante actualización que amplía sus horizontes.

La versión 1.3 permite y capacita a la aplicación de convertir cualquier archivo, sin importar su formato, en una cadena de bytes dentro de un .txt

A su vez, estos archivos .txt generados por el software son perfectamente compatibles para volver a su archivo original. El programa pregunta la extensión original del archivo con el propósito de reconstruirlo efectivamente.

De esta manera, podemos convertir imágenes, DOCs, audio, PDFs y entre otros formatos a cadenas de bytes, y luego reconstruirlos. Esta ampliación resulta fundamental para avanzar en pruebas del proyecto Col-Fi y permitir a los usuarios reconstruir sus archivos.

El nuevo código fuente se encuentra en el repositorio oficial de GitHub, no viene al caso volver a insertarlo ya que se encuentra público así como su nuevo ejecutable para ser ejecutado, por el momento, en Windows.

6. Conclusión y cierre.

Es el objetivo de este programa y extensión del proyecto agilizar y demostrar que Col-Fi puede reconstruir datos en formato bytes en información útil, real e igual a como fue enviada de forma satisfactoria y efectiva.

La interfaz gráfica de este software secundario permite ampliar el enfoque de aprendizaje hacia otras metodologías necesarias para el desarrollo del proyecto base, que en este caso explora el proceso de manipulación y conversión de archivos.